

Java AI 应用开发课程

Vibe 讲义版

参考《从 Vibe Coding 到 Harness》的长文实战复盘模式：少讲空泛概念，多讲真实卡点、判断标准、工程抓手和可复盘流程。

读这版的方式

先看故事里的问题，再看表格里的判断，最后把流程图和代码骨架画到自己的项目里。

生成日期：2026-08-09

输出目录：D:\acme\AI-Learning\codex版

写在前面：这不是另一份名词表

AI 应用开发最容易误判的一点是：以为难点都在模型。真正落地以后你会发现，难的是协作、流程、信任、权限、验证、回滚和持续优化。

这一版不追求把每个概念平均展开，而是按工程复盘的方式讲：先说一个具体问题，再把背后的知识点拆出来，最后给出 Java 开发者能落地的结构。

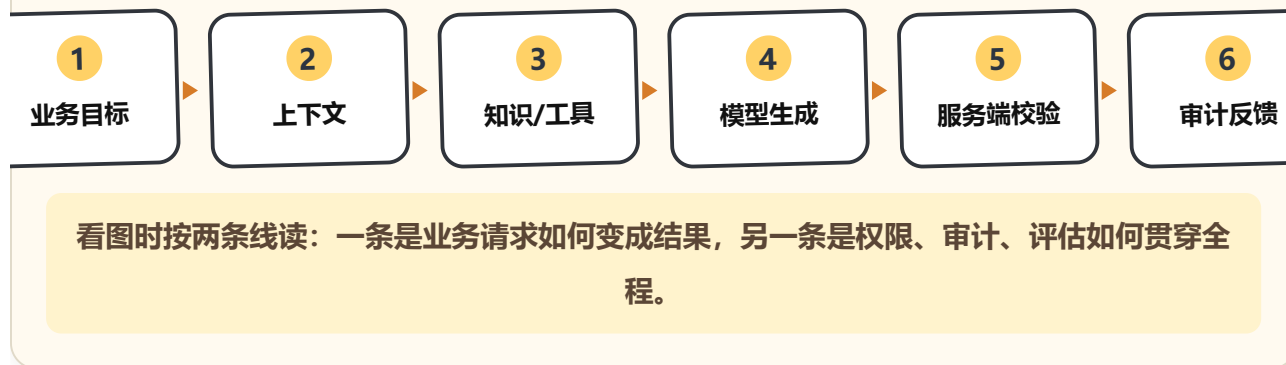
- 如果你正在从 Java 后端转 AI 应用开发，先抓住系统边界，不要先追 Agent。
- 如果你已经会调模型，下一步要学会把模型调用变成可观察、可评估、可回滚的业务能力。
- 如果你要做企业级应用，请始终问自己：事实从哪来，权限谁判断，失败怎么停，结果谁验证。

一句话：AI 工程化不是把模型接进系统，而是把不确定能力关进确定流程里。

AI 应用开发认知

建立企业级 AI 应用的总视角：模型不是系统本身，AI 应用是模型、上下文、知识、工具、流程和治理组合出来的工程系统。

AI 应用开发认知 - 一张图先建立直觉



这一章先给结论

不是模型问题，是系统边界问题。把“会调用模型”改成“能稳定交付一个业务结果”。模型只是一段能力，工程系统才负责确定性。

AI 应用开发认知：先讲一个真实卡点

团队第一次做 AI 功能时，最容易冲进模型选型、Prompt 调参、页面 Demo。跑通以后才发现真正难的是：谁给事实、谁管权限、谁确认结果、错了谁兜底。AI 应用不是一个模型接口，而是一条业务链路。

把“会调用模型”改成“能稳定交付一个业务结果”。模型只是一段能力，工程系统才负责确定性。

问题不长这样，别急着写代码

真实卡点	长什么样	工程抓手
只画模型调用	架构图里只有前端、后端、模型，缺少知识、工具、权限、审计、评估。	补数据流和控制流，标出每个组件的输入、输出和兜底方。
目标不清	上线后很难判断到底节省了什么，质量提升在哪里。	先写业务价值假设和验收指标，再做技术方案。
失败路径缺失	资料不足也回答，工具失败也继续编，用户不满意也没有回路。	为拒答、降级、转人工、重新检索设计分支。

注意，这些问题没有一个是“模型太笨”能解释完的。它们通常是系统边界、数据治理、协作流程和验证手段的问题。

AI 应用开发认知：知识点地图

这一页不是目录，而是本章的“问题索引”。每个知识点都要能回答：它解决什么问题，输入输出是什么，边界在哪里，失败时怎么处理。

知识点	先别背定义，先看它解决什么	这一章的落地判断
LLM 与传统后端的差异 medium	LLM 是概率生成系统，传统后端是确定逻辑系统。企业 AI 的核心工作，是在不确定能力外面包一层确定工程边界。	画出数据流和控制流，标出模型、知识、工具、权限、审计各自负责什么。
AI 应用六层架构 complex	企业 AI 至少包含模型层、上下文层、知识层、工具层、编排层和治理层。缺任何一层，Demo 上线后都会暴露问题。	先写验收指标，再决定模型/RAG/Tool/Agent 的组合。
Token 与上下文窗口 medium	模型看到的是 token 序列，不是无限记忆。上下文窗口决定模型一次能参考多少信息，也直接影响成本和稳定性。	为拒答、降级、转人工和反馈设计明确分支。
模型能力边界 medium	模型擅长语言理解、生成和归纳，但不天然具备企业实时事实、权限判断和业务执行能力。	画出数据流和控制流，标出模型、知识、工具、权限、审计各自负责什么。
幻觉的工程本质 complex	幻觉不是简单 bug，而是资料不足、约束不足或目标不清时，模型仍倾向生成合理文本。工程上要用 RAG、工具、拒答和评估约束。	先写验收指标，再决定模型/RAG/Tool/Agent 的组合。
AI 应用价值判断 small	不是所有功能都值得 AI 化。适合 AI 的任务通常语言密集、规则复杂、信息分散、人工成本高。	为拒答、降级、转人工和反馈设计明确分支。
Java 开发者的优势 small	Java 开发者擅长系统边界、事务、权限、稳定性和工程化，这正是企业 AI 落地最缺的能力。	画出数据流和控制流，标出模型、知识、工具、权限、审计各自负责什么。
从聊天到工作流 complex	聊天只是入口，真正的企业价值来自把回答变成流程节点、数据更新、审批建议和可追踪结论。	先写验收指标，再决定模型/RAG/Tool/Agent 的组合。
人机协作闭环 medium	AI 负责分析、生成和建议，人负责目标、判断、确认和反馈，系统负责记录过程并推动迭代。	为拒答、降级、转人工和反馈设计明确分支。
学习路线总图 small	学习要按模型调用、Prompt、Embedding、RAG、Tool、Agent、治理逐层推进，不要一开始就追复杂 Agent。	画出数据流和控制流，标出模型、知识、工具、权限、审计各自负责什么。

读法：不要孤立背概念。把每一行都放回本章流程图里，看它到底卡在哪个环节。

AI 应用开发认知：流程拆解图

复杂知识点要像工程流程一样拆：谁发起，谁理解，谁补事实，谁调用工具，谁执行，谁校验，谁反馈。



主流程

业务请求到结果返回，必须能复盘每一步。

补充流

知识、数据、工具、历史状态只在需要时进入上下文。

治理流

权限、审计、成本、评估贯穿全链路。

AI 应用开发认知：Java 落地骨架

下面的代码不是让你照抄，而是让你抓住工程边界：模型在哪，校验在哪，审计在哪，失败怎么停。

```
Goal goal = clarify(request);
AiBoundary boundary = splitModelKnowledgeTools(goal);
Context ctx = contextBuilder.build(request, boundary);
AiResult draft = model.generate(ctx);
CheckedResult result = verifier.check(draft, policy);
audit.record(request, ctx, result);
```

落地判断标准

- 业务入口不要直接调用模型，先抽应用服务或 AI Gateway。
- 模型输出只能作为候选结果，最终进入业务系统前必须经过校验。
- 日志要能回放：用户输入、上下文、模型、知识来源、工具调用、token、耗时和结果。
- 所有高风险动作都要有人类确认或明确策略。

让生产问题的人，同时生产验证手段。否则“开发完成”只是把风险从代码里转移到了用户那里。

AI 应用开发认知：学完要能自查

必须能回答

- 这个模块的输入、输出、上下游依赖分别是什么？
- 哪些事由模型负责，哪些事必须由 Java 系统负责？
- 如果资料不足、权限不足、工具失败、成本超限，系统怎么停下来？
- 怎么设计评估样本，证明这次改动真的变好？

本章小结

不是模型问题，是系统边界问题：把“会调用模型”改成“能稳定交付一个业务结果”。模型只是一段能力，工程系统才负责确定性。

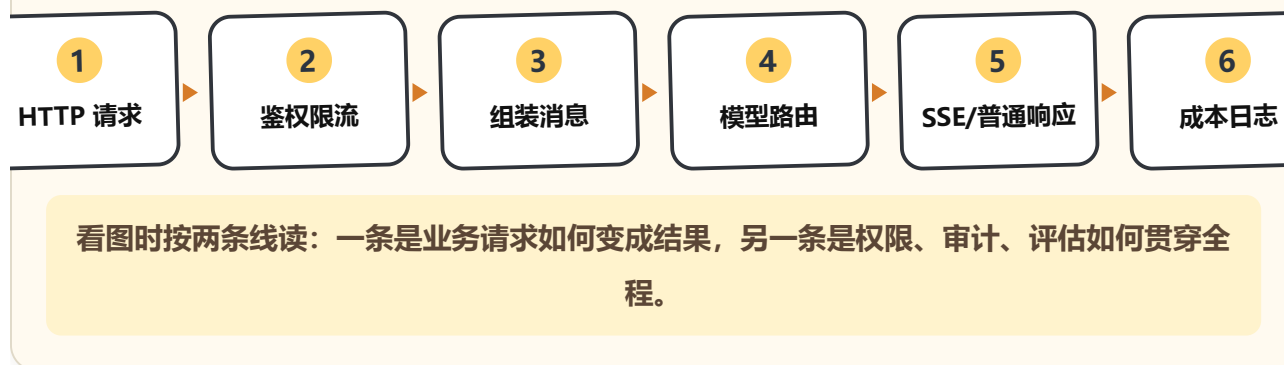
带走一个动作

把你现在项目里的一个 AI 功能画成六步链路：业务目标 -> 上下文 -> 知识/工具 -> 模型生成 -> 服务端校验 -> 审计反馈。如果画不出来，说明这个功能现在还不是一个可治理的工程能力。

模型 API 与流式聊天

把模型调用封装成稳定后端能力，重点是路由、流式、超时、fallback、成本和调用审计。

模型 API 与流式聊天 - 一张图先建立直觉



这一章先给结论

不是 HTTP 问题，是稳定性入口问题。先有 AI Gateway，再有具体模型调用。业务层只关心任务类型、成本等级、质量等级和返回契约。

模型 API 与流式聊天：先讲一个真实卡点

最早大家会在 Controller 里直接写模型 SDK。第一个 Demo 很快，第二个模型、第三个业务线、第四个降级策略一来，代码就开始散。模型 API 工程化，就是把外部不稳定能力包成内部稳定能力。

先有 AI Gateway，再有具体模型调用。业务层只关心任务类型、成本等级、质量等级和返回契约。

问题不长这样，别急着写代码

真实卡点	长什么样	工程抓手
接口散落	每个业务方法都拼 messages、配参数、处理异常。	统一 ModelClient/AiGateway，集中路由、降级、审计。
流式不可控	用户关闭页面后后台还在生成，异常事件也没记录。	SSE 增加 cancel、heartbeat、error event 和审计补记。
成本黑盒	看不到 input/output token、重试次数、fallback 率。	把 token、模型、耗时、状态码写入调用日志。

注意，这些问题没有一个是“模型太笨”能解释完的。它们通常是系统边界、数据治理、协作流程和验证手段的问题。

模型 API 与流式聊天：知识点地图

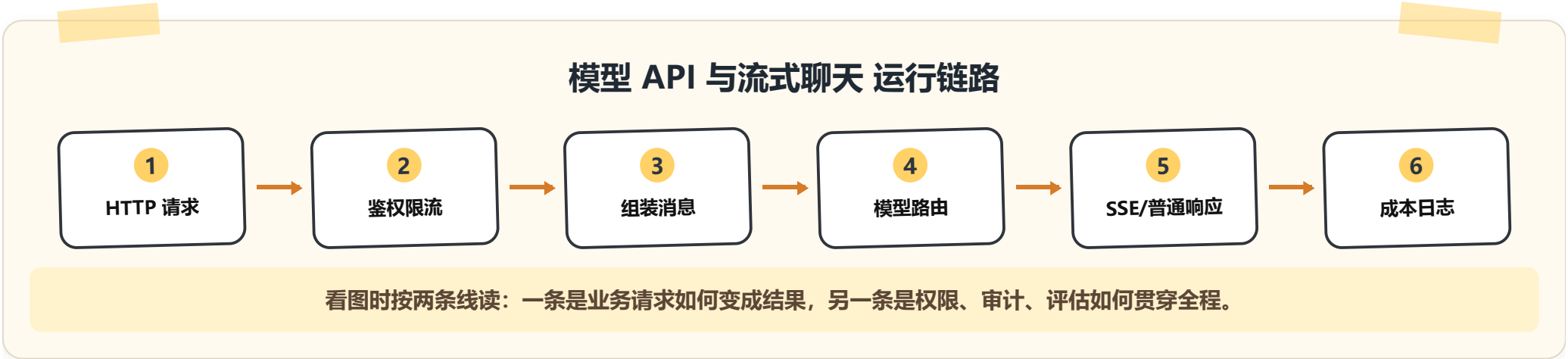
这一页不是目录，而是本章的“问题索引”。每个知识点都要能回答：它解决什么问题，输入输出是什么，边界在哪里，失败时怎么处理。

知识点	先别背定义，先看它解决什么	这一章的落地判断
Chat Completion 调用链路 complex	一次模型调用包含 system/user 消息、参数、上下文、模型选择、响应解析和错误处理，不只是一个 HTTP 请求。	统一走 AiGateway，不让业务 Controller 直接拼模型请求。
System Prompt 与 User Prompt medium	System Prompt 定义角色和边界，User Prompt 承载具体任务。两者混用会让提示词不可治理。	记录模型、token、耗时、重试、fallback 和错误码，形成调用账本。
模型参数 temperature/top_p small	参数决定输出稳定性和发散度。事实问答偏低温度，创意生成可提高。	普通调用和 SSE 流式调用都要有超时、取消和异常事件。
SSE 流式输出 complex	流式输出优化用户感知等待，但带来断线、取消、异常补偿、审计和前端状态管理复杂度。	统一走 AiGateway，不让业务 Controller 直接拼模型请求。
模型路由策略 medium	不同模型适合不同任务。企业要按质量、成本、延迟、隐私和可用性做路由。	记录模型、token、耗时、重试、fallback 和错误码，形成调用账本。
超时、重试与 fallback medium	模型服务不稳定是常态。系统必须有超时、有限重试、降级模型和友好错误。	普通调用和 SSE 流式调用都要有超时、取消和异常事件。
Token 成本核算 medium	成本由输入 token、输出 token、模型价格、重试、上下文长度和缓存命中共同决定。	统一走 AiGateway，不让业务 Controller 直接拼模型请求。
Prompt Cache small	重复问题或稳定上下文可以缓存，但必须注意权限、时效和个性化差异。	记录模型、token、耗时、重试、fallback 和错误码，形成调用账本。
OpenAI 兼容接口 small	兼容接口不代表行为完全一致，参数、流式、错误码和工具调用都要测试。	普通调用和 SSE 流式调用都要有超时、取消和异常事件。
模型调用日志 medium	日志是评估、成本、审计和安全的基础，不能只为排错记录。	统一走 AiGateway，不让业务 Controller 直接拼模型请求。

读法：不要孤立背概念。把每一行都放回本章流程图里，看它到底卡在哪个环节。

模型 API 与流式聊天：流程拆解图

复杂知识点要像工程流程一样拆：谁发起，谁理解，谁补事实，谁调用工具，谁执行，谁校验，谁反馈。



主流程

业务请求到结果返回，必须能复盘每一步。

补充流

知识、数据、工具、历史状态只在需要时进入上下文。

治理流

权限、审计、成本、评估贯穿全链路。

模型 API 与流式聊天：Java 落地骨架

下面的代码不是让你照抄，而是让你抓住工程边界：模型在哪，校验在哪，审计在哪，失败怎么停。

```
AiRequest req = requestFactory.from(httpRequest);
Route route = router.choose(req.taskType(), req.quality());
AiResponse res = aiGateway.call(req, route);
metrics.record(route.model(), res.tokens(), res.latency());
return responseMapper.toHttp(res);
```

落地判断标准

- 业务入口不要直接调用模型，先抽应用服务或 AI Gateway。
- 模型输出只能作为候选结果，最终进入业务系统前必须经过校验。
- 日志要能回放：用户输入、上下文、模型、知识来源、工具调用、token、耗时和结果。
- 所有高风险动作都要有人类确认或明确策略。

让生产问题的人，同时生产验证手段。否则“开发完成”只是把风险从代码里转移到了用户那里。

模型 API 与流式聊天：学完要能自查

必须能回答

- 这个模块的输入、输出、上下游依赖分别是什么？
- 哪些事由模型负责，哪些事必须由 Java 系统负责？
- 如果资料不足、权限不足、工具失败、成本超限，系统怎么停下来？
- 怎么设计评估样本，证明这次改动真的变好？

本章小结

不是 HTTP 问题，是稳定性入口问题：先有 AI Gateway，再有具体模型调用。业务层只关心任务类型、成本等级、质量等级和返回契约。

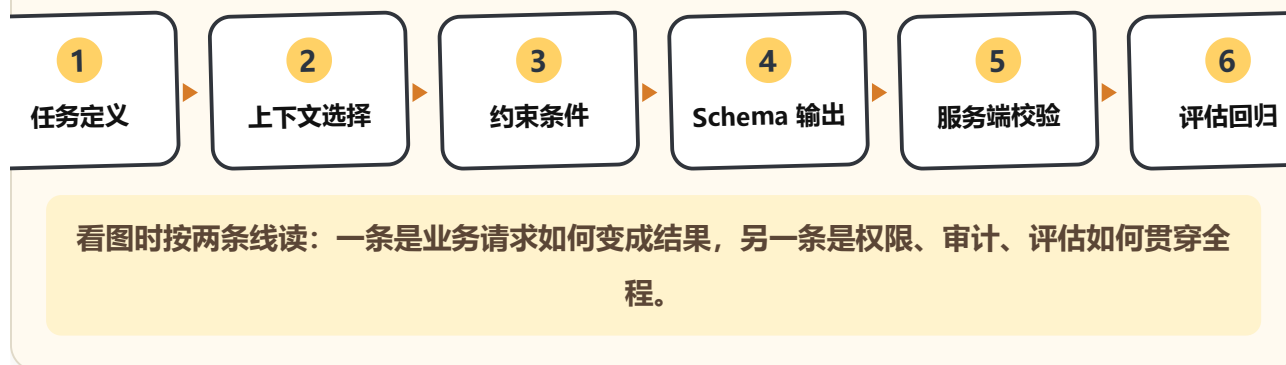
带走一个动作

把你现在项目里的一个 AI 功能画成六步链路：HTTP 请求 -> 鉴权限流 -> 组装消息 -> 模型路由 -> SSE/普通响应 -> 成本日志。如果画不出来，说明这个功能现在还不是一个可治理的工程能力。

Prompt 与结构化输出

把 Prompt 当成接口协议，让模型输出能稳定进入 Java 系统。

Prompt 与结构化输出 - 一张图先建立直觉



这一章先给结论

不是写作文，是设计接口协议。Prompt 要拆成角色、任务、上下文、约束、输出格式、失败策略，并像接口一样版本化和评估。

Prompt 与结构化输出：先讲一个真实卡点

Prompt 写得越长，团队越觉得安全。但真正出问题时，没人知道哪句话导致行为变化、哪个字段该修、哪个版本该回滚。把 Prompt 当接口协议，才有工程治理的可能。

Prompt 要拆成角色、任务、上下文、约束、输出格式、失败策略，并像接口一样版本化和评估。

问题不长这样，别急着写代码

真实卡点	长什么样	工程抓手
提示词大作文	角色、任务、规则、示例全混在一起，改一处影响全局。	拆模板字段，给每个字段明确责任。
结构化输出漂	JSON 不稳定、字段缺失、业务含义不清。	用 JSON Schema/Java record 做输出契约和校验修复。
注入风险	用户把恶意指令伪装成资料，让模型忽略规则。	输入隔离、工具权限、输出校验三层一起防。

注意，这些问题没有一个是“模型太笨”能解释完的。它们通常是系统边界、数据治理、协作流程和验证手段的问题。

Prompt 与结构化输出：知识点地图

这一页不是目录，而是本章的“问题索引”。每个知识点都要能回答：它解决什么问题，输入输出是什么，边界在哪里，失败时怎么处理。

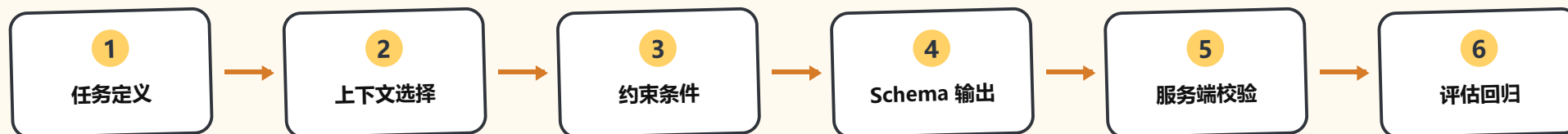
知识点	先别背定义，先看它解决什么	这一章的落地判断
Prompt 是接口协议 complex	Prompt 的作用是定义模型执行任务的协议，不是写一段求模型听话的作文。	把 Prompt 拆成角色、任务、上下文、约束、输出 Schema 和失败策略。
角色与任务拆分 medium	角色定义模型身份，任务定义要完成的工作，拆开后才可复用、测试和治理。	结构化输出必须经过 JSON Schema 或 Java record 校验。
上下文选择 medium	上下文围绕任务目标选择，不是越多越好。多余上下文会增加成本并引入干扰。	建立评估样本，Prompt 改动必须能对比和回滚。
Few-shot 示例 medium	少量高质量示例能约束输出风格和判断尺度，但示例必须覆盖边界情况。	把 Prompt 拆成角色、任务、上下文、约束、输出 Schema 和失败策略。
结构化输出 complex	结构化输出让 AI 结果进入业务系统，必须配合 JSON Schema、Java record、校验和异常处理。	结构化输出必须经过 JSON Schema 或 Java record 校验。
JSON 修复策略 medium	模型偶尔会输出非法 JSON，系统需要提取、修复、重试和降级。	建立评估样本，Prompt 改动必须能对比和回滚。
输出字段设计 medium	字段越贴近业务决策，AI 越有价值。字段要区分结论、证据、风险、缺失信息和建议动作。	把 Prompt 拆成角色、任务、上下文、约束、输出 Schema 和失败策略。
Prompt Injection complex	用户可能通过输入诱导模型忽略规则。防护要结合系统提示词、工具权限和输出校验。	结构化输出必须经过 JSON Schema 或 Java record 校验。
Prompt 版本管理 medium	Prompt 是线上逻辑，要版本化、灰度、回滚和效果对比。	建立评估样本，Prompt 改动必须能对比和回滚。
Prompt 评估集 medium	没有评估集就无法判断提示词是否变好。评估集要覆盖常见、边界和失败样本。	把 Prompt 拆成角色、任务、上下文、约束、输出 Schema 和失败策略。

读法：不要孤立背概念。把每一行都放回本章流程图里，看它到底卡在哪个环节。

Prompt 与结构化输出：流程拆解图

复杂知识点要像工程流程一样拆：谁发起，谁理解，谁补事实，谁调用工具，谁执行，谁校验，谁反馈。

Prompt 与结构化输出 运行链路



看图时按两条线读：一条是业务请求如何变成结果，另一条是权限、审计、评估如何贯穿全程。

主流程

业务请求到结果返回，必须能复盘每一步。

补充流

知识、数据、工具、历史状态只在需要时进入上下文。

治理流

权限、审计、成本、评估贯穿全链路。

Prompt 与结构化输出：Java 落地骨架

下面的代码不是让你照抄，而是让你抓住工程边界：模型在哪，校验在哪，审计在哪，失败怎么停。

```
PromptTemplate tpl = registry.load("risk-analysis", version);
String prompt = tpl.render(input, context, constraints);
JsonNode raw = model.call(prompt).json();
RiskResult result = validator.validate(raw, RiskResult.class);
evalSet.record(promptVersion, result);
```

落地判断标准

- 业务入口不要直接调用模型，先抽应用服务或 AI Gateway。
- 模型输出只能作为候选结果，最终进入业务系统前必须经过校验。
- 日志要能回放：用户输入、上下文、模型、知识来源、工具调用、token、耗时和结果。
- 所有高风险动作都要有人类确认或明确策略。

让生产问题的人，同时生产验证手段。否则“开发完成”只是把风险从代码里转移到了用户那里。

Prompt 与结构化输出：学完要能自查

必须能回答

- 这个模块的输入、输出、上下游依赖分别是什么？
- 哪些事由模型负责，哪些事必须由 Java 系统负责？
- 如果资料不足、权限不足、工具失败、成本超限，系统怎么停下来？
- 怎么设计评估样本，证明这次改动真的变好？

本章小结

不是写作文，是设计接口协议：Prompt 要拆成角色、任务、上下文、约束、输出格式、失败策略，并像接口一样版本化和评估。

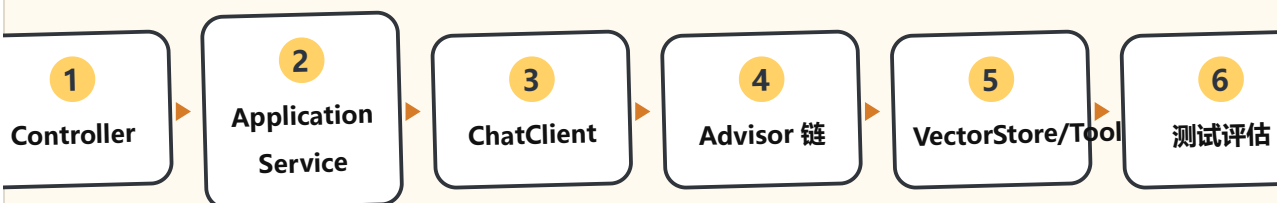
带走一个动作

把你现在项目里的一个 AI 功能画成六步链路：任务定义 -> 上下文选择 -> 约束条件 -> Schema 输出 -> 服务端校验 -> 评估回归。如果画不出来，说明这个功能现在还不是一个可治理的工程能力。

Spring AI 入门

理解 Spring AI 的工程抽象，并放入 Spring Boot 分层架构。

Spring AI 入门 - 一张图先建立直觉



看图时按两条线读：一条是业务请求如何变成结果，另一条是权限、审计、评估如何贯穿全程。

这一章先给结论

不是框架魔法，是工程抽象落位。把 AI 能力当成应用服务的一部分，而不是 Controller 里的一个模型调用片段。

Spring AI 入门：先讲一个真实卡点

Spring AI 能让调用变简单，但它不会自动帮你做权限、引用、评估和降级。框架真正的价值，是让 ChatClient、Advisor、VectorStore、Tool Callback 这些能力放到 Spring 分层里。

把 AI 能力当成应用服务的一部分，而不是 Controller 里的一个模型调用片段。

问题不长这样，别急着写代码

真实卡点	长什么样	工程抓手
Controller 过胖	HTTP 层拼 Prompt、查知识库、调模型、解析结果。	按 Controller -> Application Service -> AI Adapter 分层。
Advisor 黑盒	不知道注入了什么上下文，也不知道检索命中了哪些片段。	Advisor 链路记录上下文、来源、耗时和命中。
测试缺失	只测接口 200，不测结构化字段、拒答和权限过滤。	建立 AI 集成测试和固定评估样本。

注意，这些问题没有一个是“模型太笨”能解释完的。它们通常是系统边界、数据治理、协作流程和验证手段的问题。

Spring AI 入门：知识点地图

这一页不是目录，而是本章的“问题索引”。每个知识点都要能回答：它解决什么问题，输入输出是什么，边界在哪里，失败时怎么处理。

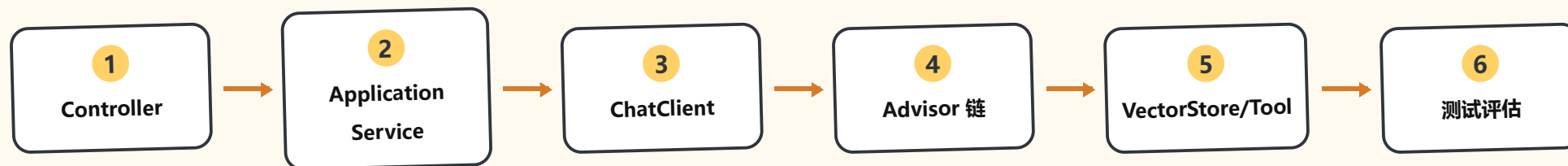
知识点	先别背定义，先看它解决什么	这一章的落地判断
ChatClient 抽象 small	ChatClient 抽象 是 Spring AI 中需要理解的工程抽象，重点是它在分层架构中的职责和边界。	把 ChatClient、Advisor、VectorStore、Tool Callback 放进 Spring 分层。
ChatClient.Builder small	ChatClient.Builder 是 Spring AI 中需要理解的工程抽象，重点是它在分层架构中的职责和边界。	Advisor 要能记录注入的上下文、命中的知识和耗时。
Model 抽象 small	Model 抽象 是 Spring AI 中需要理解的工程抽象，重点是它在分层架构中的职责和边界。	AI 集成测试要覆盖结构化解析、拒答、权限过滤和工具调用。
Advisor 机制 medium	Advisor 机制 是 Spring AI 中需要理解的工程抽象，重点是它在分层架构中的职责和边界。	把 ChatClient、Advisor、VectorStore、Tool Callback 放进 Spring 分层。
PromptTemplate small	PromptTemplate 是 Spring AI 中需要理解的工程抽象，重点是它在分层架构中的职责和边界。	Advisor 要能记录注入的上下文、命中的知识和耗时。
VectorStore 抽象 small	VectorStore 抽象 是 Spring AI 中需要理解的工程抽象，重点是它在分层架构中的职责和边界。	AI 集成测试要覆盖结构化解析、拒答、权限过滤和工具调用。
Tool Calling 集成 medium	Tool Calling 集成 是 Spring AI 中需要理解的工程抽象，重点是它在分层架构中的职责和边界。	把 ChatClient、Advisor、VectorStore、Tool Callback 放进 Spring 分层。
自动配置与排除 small	自动配置与排除 是 Spring AI 中需要理解的工程抽象，重点是它在分层架构中的职责和边界。	Advisor 要能记录注入的上下文、命中的知识和耗时。
AI 测试策略 medium	AI 测试策略 是 Spring AI 中需要理解的工程抽象，重点是它在分层架构中的职责和边界。	AI 集成测试要覆盖结构化解析、拒答、权限过滤和工具调用。
与现有项目融合 medium	与现有项目融合 是 Spring AI 中需要理解的工程抽象，重点是它在分层架构中的职责和边界。	把 ChatClient、Advisor、VectorStore、Tool Callback 放进 Spring 分层。

读法：不要孤立背概念。把每一行都放回本章流程图里，看它到底卡在哪个环节。

Spring AI 入门：流程拆解图

复杂知识点要像工程流程一样拆：谁发起，谁理解，谁补事实，谁调用工具，谁执行，谁校验，谁反馈。

Spring AI 入门 运行链路



看图时按两条线读：一条是业务请求如何变成结果，另一条是权限、审计、评估如何贯穿全程。

主流程

业务请求到结果返回，必须能复盘每一步。

补充流

知识、数据、工具、历史状态只在需要时进入上下文。

治理流

权限、审计、成本、评估贯穿全链路。

Spring AI 入门：Java 落地骨架

下面的代码不是让你照抄，而是让你抓住工程边界：模型在哪，校验在哪，审计在哪，失败怎么停。

```
@Service
class AiApplicationService {
    Answer ask(Question q) {
        return chatClient.prompt(promptFactory.create(q))
            .advisors(ragAdvisor, auditAdvisor)
            .call().entity(Answer.class);
    }
}
```

落地判断标准

- 业务入口不要直接调用模型，先抽应用服务或 AI Gateway。
- 模型输出只能作为候选结果，最终进入业务系统前必须经过校验。
- 日志要能回放：用户输入、上下文、模型、知识来源、工具调用、token、耗时和结果。
- 所有高风险动作都要有人类确认或明确策略。

让生产问题的人，同时生产验证手段。否则“开发完成”只是把风险从代码里转移到了用户那里。

Spring AI 入门：学完要能自查

必须能回答

- 这个模块的输入、输出、上下游依赖分别是什么？
- 哪些事由模型负责，哪些事必须由 Java 系统负责？
- 如果资料不足、权限不足、工具失败、成本超限，系统怎么停下来？
- 怎么设计评估样本，证明这次改动真的变好？

本章小结

不是框架魔法，是工程抽象落位：把 AI 能力当成应用服务的一部分，而不是 Controller 里的一个模型调用片段。

带走一个动作

把你现在项目里的一个 AI 功能画成六步链路：Controller -> Application Service -> ChatClient -> Advisor 链 -> VectorStore/Tool -> 测试评估。如果画不出来，说明这个功能现在还不是一个可治理的工程能力。

Embedding 与向量库

理解语义检索底层逻辑：向量模型、文本构造、索引、元数据和评估。

Embedding 与向量库 - 一张图先建立直觉



看图时按两条线读：一条是业务请求如何变成结果，另一条是权限、审计、评估如何贯穿全程。

这一章先给结论

不是相似度游戏，是语义召回工程。Embedding 只负责召回候选，生产质量来自文本构造、元数据、过滤、重排和评估。

Embedding 与向量库：先讲一个真实卡点

很多团队第一次做向量检索，都会把正文一切、向量一存、TopK 一查，然后觉得 RAG 完成了。真正上线后才发现，搜到的相似片段可能过期、无权限、不权威，甚至根本不能回答问题。

Embedding 只负责召回候选，生产质量来自文本构造、元数据、过滤、重排和评估。

问题不长这样，别急着写代码

真实卡点	长什么样	工程抓手
向量混用	不同模型生成的向量混在一个集合里。	按 embedding model/version 隔离集合。
元数据缺失	无法按租户、权限、来源、版本过滤。	元数据设计先于入库，至少保留 source、tenant、acl、version。
TopK 迷信	调大 TopK 以为更准，实际噪声更多。	用 Recall@K、MRR、人工相关性评分校准。

注意，这些问题没有一个是“模型太笨”能解释完的。它们通常是系统边界、数据治理、协作流程和验证手段的问题。

Embedding 与向量库：知识点地图

这一页不是目录，而是本章的“问题索引”。每个知识点都要能回答：它解决什么问题，输入输出是什么，边界在哪里，失败时怎么处理。

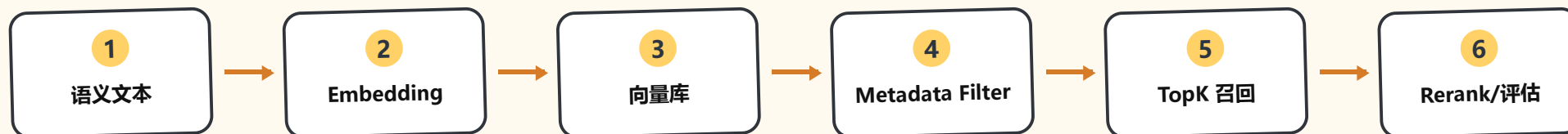
知识点	先别背定义，先看它解决什么	这一章的落地判断
Embedding 本质 complex	Embedding 把文本映射到高维向量空间，让语义相近的文本在距离上更接近。	向量集合按 embedding model/version 隔离，不混用。
向量维度 small	维度由 embedding 模型决定，不同模型向量空间不可混用。	元数据必须包含来源、权限、租户、版本和可追溯 ID。
相似度度量 medium	常见度量有 cosine、dot product、L2，必须和模型训练方式及索引匹配。	用 Recall@K、MRR、人工相关性评分调 TopK 和阈值。
语义文本构造 medium	向量化前的文本组织决定召回质量，要把业务字段合成模型能理解的语义画像。	向量集合按 embedding model/version 隔离，不混用。
Chunk 策略 complex	chunk 影响召回粒度。太大干扰多，太小语义不足，要按标题、段落和语义边界切。	元数据必须包含来源、权限、租户、版本和可追溯 ID。
Metadata 设计 complex	元数据用于权限、过滤、溯源和版本治理，是生产级向量库关键。	用 Recall@K、MRR、人工相关性评分调 TopK 和阈值。
pgvector medium	pgvector 适合 Java 企业项目快速落地，能和 PostgreSQL 事务、SQL、权限结合。	向量集合按 embedding model/version 隔离，不混用。
Milvus/Redis Vector small	Milvus 偏大规模检索，Redis Vector 偏低延迟和缓存场景。	元数据必须包含来源、权限、租户、版本和可追溯 ID。
TopK 与阈值 medium	TopK 不是越大越好，阈值要通过评估集校准。	用 Recall@K、MRR、人工相关性评分调 TopK 和阈值。
语义搜索评估 medium	评估要看是否搜到正确答案、是否排序靠前、是否带正确元数据。	向量集合按 embedding model/version 隔离，不混用。

读法：不要孤立背概念。把每一行都放回本章流程图里，看它到底卡在哪个环节。

Embedding 与向量库：流程拆解图

复杂知识点要像工程流程一样拆：谁发起，谁理解，谁补事实，谁调用工具，谁执行，谁校验，谁反馈。

Embedding 与向量库 运行链路



看图时按两条线读：一条是业务请求如何变成结果，另一条是权限、审计、评估如何贯穿全程。

主流程

业务请求到结果返回，必须能复盘每一步。

补充流

知识、数据、工具、历史状态只在需要时进入上下文。

治理流

权限、审计、成本、评估贯穿全链路。

Embedding 与向量库：Java 落地骨架

下面的代码不是让你照抄，而是让你抓住工程边界：模型在哪，校验在哪，审计在哪，失败怎么停。

```
String text = semanticTextBuilder.build(doc);
float[] vector = embeddingModel.embed(text);
vectorStore.upsert(doc.id(), vector, metadata);
List<Match> hits = vectorStore.search(queryVector, filter, topK);
List<Match> ranked = reranker.rank(query, hits);
```

落地判断标准

- 业务入口不要直接调用模型，先抽应用服务或 AI Gateway。
- 模型输出只能作为候选结果，最终进入业务系统前必须经过校验。
- 日志要能回放：用户输入、上下文、模型、知识来源、工具调用、token、耗时和结果。
- 所有高风险动作都要有人类确认或明确策略。

让生产问题的人，同时生产验证手段。否则“开发完成”只是把风险从代码里转移到了用户那里。

Embedding 与向量库：学完要能自查

必须能回答

- 这个模块的输入、输出、上下游依赖分别是什么？
- 哪些事由模型负责，哪些事必须由 Java 系统负责？
- 如果资料不足、权限不足、工具失败、成本超限，系统怎么停下来？
- 怎么设计评估样本，证明这次改动真的变好？

本章小结

不是相似度游戏，是语义召回工程：Embedding 只负责召回候选，生产质量来自文本构造、元数据、过滤、重排和评估。

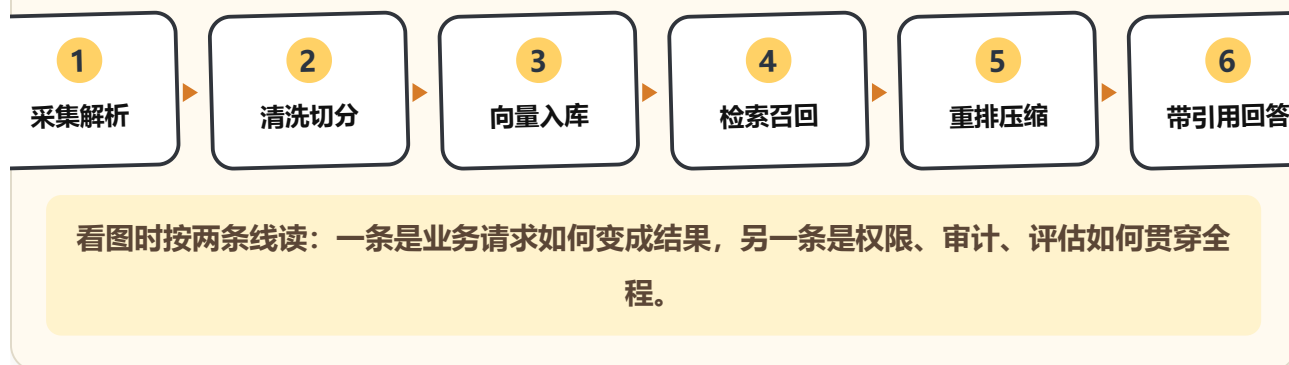
带走一个动作

把你现在项目里的一个 AI 功能画成六步链路：语义文本 -> Embedding -> 向量库 -> Metadata Filter -> TopK 召回 -> Rerank/评估。如果画不出来，说明这个功能现在还不是一个可治理的工程能力。

RAG 知识库 v1

搭建最小可用 RAG 链路：采集、解析、切分、检索、引用和拒答。

RAG 知识库 v1 - 一张图先建立直觉



这一章先给结论

不是向量库加模型，是证据流水线。把 RAG 当成“证据生产线”：离线生产可信片段，在线选择证据，生成时必须带引用和拒答条件。

RAG 知识库 v1：先讲一个真实卡点

RAG 的坑通常不在回答那一刻，而在更早的资料治理：文档解析错了、chunk 切断了、权限没带上、来源不可追溯。最后模型只是把这些问题用流畅中文暴露出来。

把 RAG 当成“证据生产线”：离线生产可信片段，在线选择证据，生成时必须带引用和拒答条件。

问题不长这样，别急着写代码

真实卡点	长什么样	工程抓手
chunk 粗糙	整页、整章、固定字数切分，标题和表格语义丢失。	按标题层级、段落、表格、条款分别切。
无引用	答案看起来正确，但无法点回原文验证。	回答对象必须携带 sourceId、chunkId、version。
资料不足强答	没检索到证据时仍然生成。	低置信度、冲突证据、无命中时拒答并说明缺什么。

注意，这些问题没有一个是“模型太笨”能解释完的。它们通常是系统边界、数据治理、协作流程和验证手段的问题。

RAG 知识库 v1：知识点地图

这一页不是目录，而是本章的“问题索引”。每个知识点都要能回答：它解决什么问题，输入输出是什么，边界在哪里，失败时怎么处理。

知识点	先别背定义，先看它解决什么	这一章的落地判断
RAG 第一性原理 complex	RAG 第一性原理 是 RAG 链路中的关键环节，决定系统能否基于可信事实回答。	离线链路先保证文档解析、chunk、元数据和版本正确。
文档采集 medium	文档采集 是 RAG 链路中的关键环节，决定系统能否基于可信事实回答。	在线链路要做检索、过滤、重排、压缩和带引用生成。
文档解析 medium	文档解析 是 RAG 链路中的关键环节，决定系统能否基于可信事实回答。	无证据、低置信或冲突证据时拒答，并说明缺什么资料。
内容清洗 medium	内容清洗 是 RAG 链路中的关键环节，决定系统能否基于可信事实回答。	离线链路先保证文档解析、chunk、元数据和版本正确。
切分与重叠 complex	切分与重叠 是 RAG 链路中的关键环节，决定系统能否基于可信事实回答。	在线链路要做检索、过滤、重排、压缩和带引用生成。
检索器 Retriever complex	检索器 Retriever 是 RAG 链路中的关键环节，决定系统能否基于可信事实回答。	无证据、低置信或冲突证据时拒答，并说明缺什么资料。
上下文组装 complex	上下文组装 是 RAG 链路中的关键环节，决定系统能否基于可信事实回答。	离线链路先保证文档解析、chunk、元数据和版本正确。
引用来源 complex	引用来源 是 RAG 链路中的关键环节，决定系统能否基于可信事实回答。	在线链路要做检索、过滤、重排、压缩和带引用生成。
资料不足拒答 complex	资料不足拒答 是 RAG 链路中的关键环节，决定系统能否基于可信事实回答。	无证据、低置信或冲突证据时拒答，并说明缺什么资料。
增量更新 medium	增量更新 是 RAG 链路中的关键环节，决定系统能否基于可信事实回答。	离线链路先保证文档解析、chunk、元数据和版本正确。

读法：不要孤立背概念。把每一行都放回本章流程图里，看它到底卡在哪个环节。

RAG 知识库 v1：流程拆解图

复杂知识点要像工程流程一样拆：谁发起，谁理解，谁补事实，谁调用工具，谁执行，谁校验，谁反馈。



RAG 知识库 v1: Java 落地骨架

下面的代码不是让你照抄，而是让你抓住工程边界：模型在哪，校验在哪，审计在哪，失败怎么停。

```
String rewritten = queryRewriter.rewrite(question);
List<Chunk> candidates = retriever.search(rewritten, aclFilter);
List<Chunk> evidence = reranker.rank(question, candidates).take(5);
Answer answer = generator.answer(question, cite(evidence));
groundedness.check(answer, evidence);
```

落地判断标准

- 业务入口不要直接调用模型，先抽应用服务或 AI Gateway。
- 模型输出只能作为候选结果，最终进入业务系统前必须经过校验。
- 日志要能回放：用户输入、上下文、模型、知识来源、工具调用、token、耗时和结果。
- 所有高风险动作都要有人类确认或明确策略。

让生产问题的人，同时生产验证手段。否则“开发完成”只是把风险从代码里转移到了用户那里。

RAG 知识库 v1：学完要能自查

必须能回答

- 这个模块的输入、输出、上下游依赖分别是什么？
- 哪些事由模型负责，哪些事必须由 Java 系统负责？
- 如果资料不足、权限不足、工具失败、成本超限，系统怎么停下来？
- 怎么设计评估样本，证明这次改动真的变好？

本章小结

不是向量库加模型，是证据流水线：把 RAG 当成“证据生产线”：离线生产可信片段，在线选择证据，生成时必须带引用和拒答条件。

带走一个动作

把你现在项目里的一个 AI 功能画成六步链路：采集解析 -> 清洗切分 -> 向量入库 -> 检索召回 -> 重排压缩 -> 带引用回答。如果画不出来，说明这个功能现在还不是一个可治理的工程能力。

RAG 工程优化

把能用的 RAG 变成更准、更稳、更可解释的 RAG。

RAG 工程优化 - 一张图先建立直觉



看图时按两条线读：一条是业务请求如何变成结果，另一条是权限、审计、评估如何贯穿全程。

这一章先给结论

不是向量库加模型，是证据流水线。把 RAG 当成“证据生产线”：离线生产可信片段，在线选择证据，生成时必须带引用和拒答条件。

RAG 工程优化：先讲一个真实卡点

RAG 的坑通常不在回答那一刻，而在更早的资料治理：文档解析错了、chunk 切断了、权限没带上、来源不可追溯。最后模型只是把这些问题用流畅中文暴露出来。

把 RAG 当成“证据生产线”：离线生产可信片段，在线选择证据，生成时必须带引用和拒答条件。

问题不长这样，别急着写代码

真实卡点	长什么样	工程抓手
chunk 粗糙	整页、整章、固定字数切分，标题和表格语义丢失。	按标题层级、段落、表格、条款分别切。
无引用	答案看起来正确，但无法点回原文验证。	回答对象必须携带 sourceId、chunkId、version。
资料不足强答	没检索到证据时仍然生成。	低置信度、冲突证据、无命中时拒答并说明缺什么。

注意，这些问题没有一个是“模型太笨”能解释完的。它们通常是系统边界、数据治理、协作流程和验证手段的问题。

RAG 工程优化：知识点地图

这一页不是目录，而是本章的“问题索引”。每个知识点都要能回答：它解决什么问题，输入输出是什么，边界在哪里，失败时怎么处理。

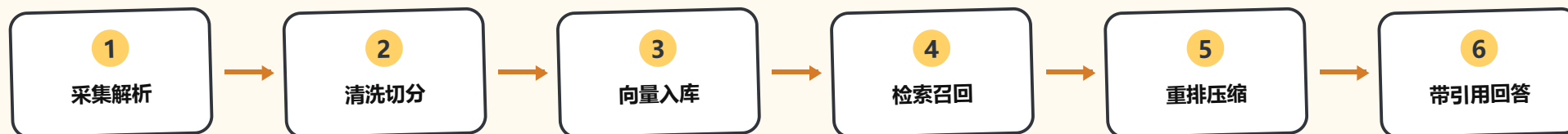
知识点	先别背定义，先看它解决什么	这一章的落地判断
Query Rewrite medium	Query Rewrite 是 RAG 从 Demo 走向生产的优化能力，重点是召回、排序、压缩、权限和评估。	离线链路先保证文档解析、chunk、元数据和版本正确。
混合检索 complex	混合检索 是 RAG 从 Demo 走向生产的优化能力，重点是召回、排序、压缩、权限和评估。	在线链路要做检索、过滤、重排、压缩和带引用生成。
Rerank complex	Rerank 是 RAG 从 Demo 走向生产的优化能力，重点是召回、排序、压缩、权限和评估。	无证据、低置信或冲突证据时拒答，并说明缺什么资料。
Metadata Filter medium	Metadata Filter 是 RAG 从 Demo 走向生产的优化能力，重点是召回、排序、压缩、权限和评估。	离线链路先保证文档解析、chunk、元数据和版本正确。
Parent-child Chunk complex	Parent-child Chunk 是 RAG 从 Demo 走向生产的优化能力，重点是召回、排序、压缩、权限和评估。	在线链路要做检索、过滤、重排、压缩和带引用生成。
Context Compression medium	Context Compression 是 RAG 从 Demo 走向生产的优化能力，重点是召回、排序、压缩、权限和评估。	无证据、低置信或冲突证据时拒答，并说明缺什么资料。
Table RAG complex	Table RAG 是 RAG 从 Demo 走向生产的优化能力，重点是召回、排序、压缩、权限和评估。	离线链路先保证文档解析、chunk、元数据和版本正确。
Graph RAG complex	Graph RAG 是 RAG 从 Demo 走向生产的优化能力，重点是召回、排序、压缩、权限和评估。	在线链路要做检索、过滤、重排、压缩和带引用生成。
RAG 评估集 complex	RAG 评估集 是 RAG 从 Demo 走向生产的优化能力，重点是召回、排序、压缩、权限和评估。	无证据、低置信或冲突证据时拒答，并说明缺什么资料。
线上反馈闭环 complex	线上反馈闭环 是 RAG 从 Demo 走向生产的优化能力，重点是召回、排序、压缩、权限和评估。	离线链路先保证文档解析、chunk、元数据和版本正确。

读法：不要孤立背概念。把每一行都放回本章流程图里，看它到底卡在哪个环节。

RAG 工程优化：流程拆解图

复杂知识点要像工程流程一样拆：谁发起，谁理解，谁补事实，谁调用工具，谁执行，谁校验，谁反馈。

RAG 工程优化 运行链路



看图时按两条线读：一条是业务请求如何变成结果，另一条是权限、审计、评估如何贯穿全程。

主流程

业务请求到结果返回，必须能复盘每一步。

补充流

知识、数据、工具、历史状态只在需要时进入上下文。

治理流

权限、审计、成本、评估贯穿全链路。

RAG 工程优化：Java 落地骨架

下面的代码不是让你照抄，而是让你抓住工程边界：模型在哪，校验在哪，审计在哪，失败怎么停。

```
String rewritten = queryRewriter.rewrite(question);
List<Chunk> candidates = retriever.search(rewritten, aclFilter);
List<Chunk> evidence = reranker.rank(question, candidates).take(5);
Answer answer = generator.answer(question, cite(evidence));
groundedness.check(answer, evidence);
```

落地判断标准

- 业务入口不要直接调用模型，先抽应用服务或 AI Gateway。
- 模型输出只能作为候选结果，最终进入业务系统前必须经过校验。
- 日志要能回放：用户输入、上下文、模型、知识来源、工具调用、token、耗时和结果。
- 所有高风险动作都要有人类确认或明确策略。

让生产问题的人，同时生产验证手段。否则“开发完成”只是把风险从代码里转移到了用户那里。

RAG 工程优化：学完要能自查

必须能回答

- 这个模块的输入、输出、上下游依赖分别是什么？
- 哪些事由模型负责，哪些事必须由 Java 系统负责？
- 如果资料不足、权限不足、工具失败、成本超限，系统怎么停下来？
- 怎么设计评估样本，证明这次改动真的变好？

本章小结

不是向量库加模型，是证据流水线：把 RAG 当成“证据生产线”：离线生产可信片段，在线选择证据，生成时必须带引用和拒答条件。

带走一个动作

把你现在项目里的一个 AI 功能画成六步链路：采集解析 -> 清洗切分 -> 向量入库 -> 检索召回 -> 重排压缩 -> 带引用回答。如果画不出来，说明这个功能现在还不是一个可治理的工程能力。

Tool Calling

让模型调用 Java 业务能力，但执行权永远在应用系统手里。

Tool Calling - 一张图先建立直觉



看图时按两条线读：一条是业务请求如何变成结果，另一条是权限、审计、评估如何贯穿全程。

这一章先给结论

不是模型有手脚，是系统给它受控能力。工具越小、越只读、越单职责，越适合交给模型调用；写操作和高风险动作必须进入人工确认。

Tool Calling：先讲一个真实卡点

Tool Calling 让模型看起来能查库、调接口、读文件，但真正的执行权不能交给模型。模型只提出意图，应用系统必须做参数、权限、风险、审计和人工确认。

工具越小、越只读、越单职责，越适合交给模型调用；写操作和高风险动作必须进入人工确认。

问题不长这样，别急着写代码

真实卡点	长什么样	工程抓手
工具过大	把内部 Service 整个暴露给模型。	拆成单职责小工具，并限制参数范围。
参数直信	模型生成 SQL、ID、金额、状态后直接执行。	Java 后端校验枚举、权限、租户、幂等和风险。
MCP 误解	以为 MCP 只是另一种工具调用写法。	理解 Host/Client/Server，让工具一次实现多处复用。

注意，这些问题没有一个是“模型太笨”能解释完的。它们通常是系统边界、数据治理、协作流程和验证手段的问题。

Tool Calling：知识点地图

这一页不是目录，而是本章的“问题索引”。每个知识点都要能回答：它解决什么问题，输入输出是什么，边界在哪里，失败时怎么处理。

知识点	先别背定义，先看它解决什么	这一章的落地判断
Tool Calling 本质 complex	Tool Calling 本质 负责把外部能力安全暴露给模型，同时保持权限、校验和审计边界。	工具要小、只读、单职责，参数用 schema 和枚举约束。
工具 Schema complex	工具 Schema 负责把外部能力安全暴露给模型，同时保持权限、校验和审计边界。	模型只生成工具调用意图，Java 后端负责权限、校验和执行。
参数校验 complex	参数校验 负责把外部能力安全暴露给模型，同时保持权限、校验和审计边界。	高风险写操作必须人工确认，所有工具调用都要审计。
只读优先 medium	只读优先 负责把外部能力安全暴露给模型，同时保持权限、校验和审计边界。	工具要小、只读、单职责，参数用 schema 和枚举约束。
工具结果摘要 medium	工具结果摘要 负责把外部能力安全暴露给模型，同时保持权限、校验和审计边界。	模型只生成工具调用意图，Java 后端负责权限、校验和执行。
工具审计 medium	工具审计 负责把外部能力安全暴露给模型，同时保持权限、校验和审计边界。	高风险写操作必须人工确认，所有工具调用都要审计。
MCP Server complex	MCP Server 负责把外部能力安全暴露给模型，同时保持权限、校验和审计边界。	工具要小、只读、单职责，参数用 schema 和枚举约束。
Tool Registry complex	Tool Registry 负责把外部能力安全暴露给模型，同时保持权限、校验和审计边界。	模型只生成工具调用意图，Java 后端负责权限、校验和执行。
工具失败恢复 complex	工具失败恢复 负责把外部能力安全暴露给模型，同时保持权限、校验和审计边界。	高风险写操作必须人工确认，所有工具调用都要审计。
Human Confirmation complex	Human Confirmation 负责把外部能力安全暴露给模型，同时保持权限、校验和审计边界。	工具要小、只读、单职责，参数用 schema 和枚举约束。

读法：不要孤立背概念。把每一行都放回本章流程图里，看它到底卡在哪个环节。

Tool Calling: 流程拆解图

复杂知识点要像工程流程一样拆：谁发起，谁理解，谁补事实，谁调用工具，谁执行，谁校验，谁反馈。



Tool Calling: Java 落地骨架

下面的代码不是让你照抄，而是让你抓住工程边界：模型在哪，校验在哪，审计在哪，失败怎么停。

```
ToolCall call = model.decideTool(intent, toolSchemas);
Args args = argValidator.validate(call.args(), user);
PolicyDecision pd = policy.check(user, call.tool(), args);
ToolResult result = executor.executeReadOnly(pd, args);
audit.save(call, args.digest(), result.summary());
```

落地判断标准

- 业务入口不要直接调用模型，先抽应用服务或 AI Gateway。
- 模型输出只能作为候选结果，最终进入业务系统前必须经过校验。
- 日志要能回放：用户输入、上下文、模型、知识来源、工具调用、token、耗时和结果。
- 所有高风险动作都要有人类确认或明确策略。

让生产问题的人，同时生产验证手段。否则“开发完成”只是把风险从代码里转移到了用户那里。

Tool Calling：学完要能自查

必须能回答

- 这个模块的输入、输出、上下游依赖分别是什么？
- 哪些事由模型负责，哪些事必须由 Java 系统负责？
- 如果资料不足、权限不足、工具失败、成本超限，系统怎么停下来？
- 怎么设计评估样本，证明这次改动真的变好？

本章小结

不是模型有手脚，是系统给它受控能力：工具越小、越只读、越单职责，越适合交给模型调用；写操作和高风险动作必须进入人工确认。

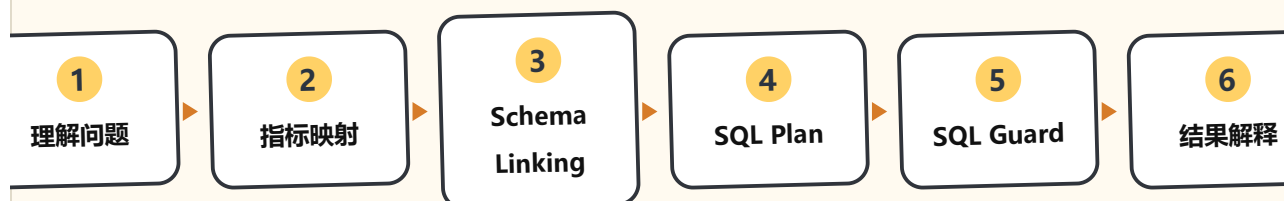
带走一个动作

把你现在项目里的一个 AI 功能画成六步链路：用户任务 -> 工具选择 -> 参数生成 -> 服务端校验 -> 工具执行 -> 观察结果。如果画不出来，说明这个功能现在还不是一个可治理的工程能力。

数据库问答助手

NL2SQL 价值高也风险高，必须用语义层、安全层和审计层包住。

数据库问答助手 - 一张图先建立直觉



看图时按两条线读：一条是业务请求如何变成结果，另一条是权限、审计、评估如何贯穿全程。

这一章先给结论

不是自然语言转 SQL，是安全数据问答。自然语言先转业务指标和查询计划，再由白名单 schema 渲染 SQL，最后通过 SQL Guard 执行。

数据库问答助手：先讲一个真实卡点

NL2SQL 的演示很容易惊艳：一句话生成 SQL。上线时真正吓人：模型猜表、猜字段、越权查敏感数据、没 LIMIT 跑满数据库。数据库问答必须先建语义层和安全层。

自然语言先转业务指标和查询计划，再由白名单 schema 渲染 SQL，最后通过 SQL Guard 执行。

问题不长这样，别急着写代码

真实卡点	长什么样	工程抓手
自由猜表	模型凭字段名语感拼 SQL。	用指标字典、表字段白名单、Schema Linking 约束。
高权限执行	模型生成什么就执行什么。	只读账号、限定 schema、禁止危险关键字。
结果泄露	把明细、手机号、身份证等敏感字段返回。	字段策略、角色脱敏、聚合优先。

注意，这些问题没有一个是“模型太笨”能解释完的。它们通常是系统边界、数据治理、协作流程和验证手段的问题。

数据库问答助手：知识点地图

这一页不是目录，而是本章的“问题索引”。每个知识点都要能回答：它解决什么问题，输入输出是什么，边界在哪里，失败时怎么处理。

知识点	先别背定义，先看它解决什么	这一章的落地判断
NL2SQL 本质 complex	NL2SQL 本质 是安全数据问答能力中的关键知识点，不能只追求生成 SQL。	先识别业务指标和查询计划，再生成 SQL。
语义层 complex	语义层 是安全数据问答能力中的关键知识点，不能只追求生成 SQL。	Schema Linking 只允许绑定白名单表字段。
Schema Linking complex	Schema Linking 是安全数据问答能力中的关键知识点，不能只追求生成 SQL。	SQL Guard 要检查只读、LIMIT、超时、敏感字段和危险关键字。
SQL 生成计划 medium	SQL 生成计划 是安全数据问答能力中的关键知识点，不能只追求生成 SQL。	先识别业务指标和查询计划，再生成 SQL。
SQL Guard complex	SQL Guard 是安全数据问答能力中的关键知识点，不能只追求生成 SQL。	Schema Linking 只允许绑定白名单表字段。
只读账号 medium	只读账号 是安全数据问答能力中的关键知识点，不能只追求生成 SQL。	SQL Guard 要检查只读、LIMIT、超时、敏感字段和危险关键字。
分页和超时 medium	分页和超时 是安全数据问答能力中的关键知识点，不能只追求生成 SQL。	先识别业务指标和查询计划，再生成 SQL。
结果解释 medium	结果解释 是安全数据问答能力中的关键知识点，不能只追求生成 SQL。	Schema Linking 只允许绑定白名单表字段。
敏感数据脱敏 complex	敏感数据脱敏 是安全数据问答能力中的关键知识点，不能只追求生成 SQL。	SQL Guard 要检查只读、LIMIT、超时、敏感字段和危险关键字。
NL2SQL 评估 complex	NL2SQL 评估 是安全数据问答能力中的关键知识点，不能只追求生成 SQL。	先识别业务指标和查询计划，再生成 SQL。

读法：不要孤立背概念。把每一行都放回本章流程图里，看它到底卡在哪个环节。

数据库问答助手：流程拆解图

复杂知识点要像工程流程一样拆：谁发起，谁理解，谁补事实，谁调用工具，谁执行，谁校验，谁反馈。



主流程

业务请求到结果返回，必须能复盘每一步。

补充流

知识、数据、工具、历史状态只在需要时进入上下文。

治理流

权限、审计、成本、评估贯穿全链路。

数据库问答助手：Java 落地骨架

下面的代码不是让你照抄，而是让你抓住工程边界：模型在哪，校验在哪，审计在哪，失败怎么停。

```
Intent intent = semanticLayer.parse(question);
SqlPlan plan = planner.plan(intent, metricDict);
String sql = renderer.render(plan, tableWhitelist);
sqlGuard.assertSafe(sql).assertLimit().assertReadOnly();
ResultSet rows = readOnlyJdbc.query(sql);
```

落地判断标准

- 业务入口不要直接调用模型，先抽应用服务或 AI Gateway。
- 模型输出只能作为候选结果，最终进入业务系统前必须经过校验。
- 日志要能回放：用户输入、上下文、模型、知识来源、工具调用、token、耗时和结果。
- 所有高风险动作都要有人类确认或明确策略。

让生产问题的人，同时生产验证手段。否则“开发完成”只是把风险从代码里转移到了用户那里。

数据库问答助手：学完要能自查

必须能回答

- 这个模块的输入、输出、上下游依赖分别是什么？
- 哪些事由模型负责，哪些事必须由 Java 系统负责？
- 如果资料不足、权限不足、工具失败、成本超限，系统怎么停下来？
- 怎么设计评估样本，证明这次改动真的变好？

本章小结

不是自然语言转 SQL，是安全数据问答：自然语言先转业务指标和查询计划，再由白名单 schema 渲染 SQL，最后通过 SQL Guard 执行。

带走一个动作

把你现在项目里的一个 AI 功能画成六步链路：理解问题 -> 指标映射 -> Schema Linking -> SQL Plan -> SQL Guard -> 结果解释。如果画不出来，说明这个功能现在还不是一个可治理的工程能力。

Agent 工作流

Agent 是模型、工具、记忆、状态、规划、终止条件和护栏组成的受控执行系统。

Agent 工作流 - 一张图先建立直觉



这一章先给结论

不是万能机器人，是受控运行时。先做固定 Workflow Agent，再做 ReAct；先把状态、工具、终止条件跑稳，再谈 Multi-Agent。

Agent workflows: 先讲一个真实卡点

Agent 最迷人的地方是它会自己拆任务、调工具、继续思考；最危险的地方也是这里。没有最大步数、状态回放、工具分级和人工确认，Agent 会把小问题跑成大数据。

先做固定 Workflow Agent，再做 ReAct；先把状态、工具、终止条件跑稳，再谈 Multi-Agent。

问题不长这样，别急着写代码

真实卡点	长什么样	工程抓手
无限循环	模型不断反思、不断调用工具。	设置 maxSteps、maxToolCalls、timeout、终止条件。
状态不可回放	出了问题不知道为什么调用某工具。	State 记录目标、计划、工具结果、中间答案、终止原因。
权限过大	Agent 自动执行高风险动作。	工具分级，高风险必须 Human Confirmation。

注意，这些问题没有一个是“模型太笨”能解释完的。它们通常是系统边界、数据治理、协作流程和验证手段的问题。

Agent workflows: 知识点地图

这一页不是目录，而是本章的“问题索引”。每个知识点都要能回答：它解决什么问题，输入输出是什么，边界在哪里，失败时怎么处理。

知识点	先别背定义，先看它解决什么	这一章的落地判断
Agent 本质 complex	Agent 本质 是 Agent 系统的核心组成，要按运行流程、状态变化、工具边界和安全护栏讲清楚。	先做 Workflow Agent，再逐步引入 ReAct 和 Multi-Agent。
ReAct 模式 complex	ReAct 模式 是 Agent 系统的核心组成，要按运行流程、状态变化、工具边界和安全护栏讲清楚。	State 要记录目标、计划、工具结果、中间答案和终止原因。
Planning 规划 complex	Planning 规划 是 Agent 系统的核心组成，要按运行流程、状态变化、工具边界和安全护栏讲清楚。	必须设置 maxSteps、maxToolCalls、timeout 和 Human Confirmation。
Memory 记忆 complex	Memory 记忆 是 Agent 系统的核心组成，要按运行流程、状态变化、工具边界和安全护栏讲清楚。	先做 Workflow Agent，再逐步引入 ReAct 和 Multi-Agent。
State 状态 complex	State 状态 是 Agent 系统的核心组成，要按运行流程、状态变化、工具边界和安全护栏讲清楚。	State 要记录目标、计划、工具结果、中间答案和终止原因。
ToolNode complex	ToolNode 是 Agent 系统的核心组成，要按运行流程、状态变化、工具边界和安全护栏讲清楚。	必须设置 maxSteps、maxToolCalls、timeout 和 Human Confirmation。
Multi-Agent 协作 complex	Multi-Agent 协作 是 Agent 系统的核心组成，要按运行流程、状态变化、工具边界和安全护栏讲清楚。	先做 Workflow Agent，再逐步引入 ReAct 和 Multi-Agent。
终止条件 complex	终止条件 是 Agent 系统的核心组成，要按运行流程、状态变化、工具边界和安全护栏讲清楚。	State 要记录目标、计划、工具结果、中间答案和终止原因。
Agent 安全 complex	Agent 安全 是 Agent 系统的核心组成，要按运行流程、状态变化、工具边界和安全护栏讲清楚。	必须设置 maxSteps、maxToolCalls、timeout 和 Human Confirmation。
企业 Agent 流程 complex	企业 Agent 流程 是 Agent 系统的核心组成，要按运行流程、状态变化、工具边界和安全护栏讲清楚。	先做 Workflow Agent，再逐步引入 ReAct 和 Multi-Agent。

读法：不要孤立背概念。把每一行都放回本章流程图里，看它到底卡在哪个环节。

Agent 工作流：流程拆解图

复杂知识点要像工程流程一样拆：谁发起，谁理解，谁补事实，谁调用工具，谁执行，谁校验，谁反馈。



主流程

业务请求到结果返回，必须能复盘每一步。

补充流

知识、数据、工具、历史状态只在需要时进入上下文。

治理流

权限、审计、成本、评估贯穿全链路。

Agent workflows: Java 落地骨架

下面的代码不是让你照抄，而是让你抓住工程边界：模型在哪，校验在哪，审计在哪，失败怎么停。

```
while (!state.done() && state.steps() < MAX_STEPS) {  
    Decision d = llm.plan(state, allowedTools);  
    Observation o = toolNode.executeIfAllowed(d, policy);  
    state = reducer.merge(state, d, o);  
}  
return finalizer.answer(state);
```

落地判断标准

- 业务入口不要直接调用模型，先抽应用服务或 AI Gateway。
- 模型输出只能作为候选结果，最终进入业务系统前必须经过校验。
- 日志要能回放：用户输入、上下文、模型、知识来源、工具调用、token、耗时和结果。
- 所有高风险动作都要有人类确认或明确策略。

让生产问题的人，同时生产验证手段。否则“开发完成”只是把风险从代码里转移到了用户那里。

Agent workflow: 学完要能自查

必须能回答

- 这个模块的输入、输出、上下游依赖分别是什么？
- 哪些事由模型负责，哪些事必须由 Java 系统负责？
- 如果资料不足、权限不足、工具失败、成本超限，系统怎么停下来？
- 怎么设计评估样本，证明这次改动真的变好？

本章小结

不是万能机器人，是受控运行时：先做固定 Workflow Agent，再做 ReAct；先把状态、工具、终止条件跑稳，再谈 Multi-Agent。

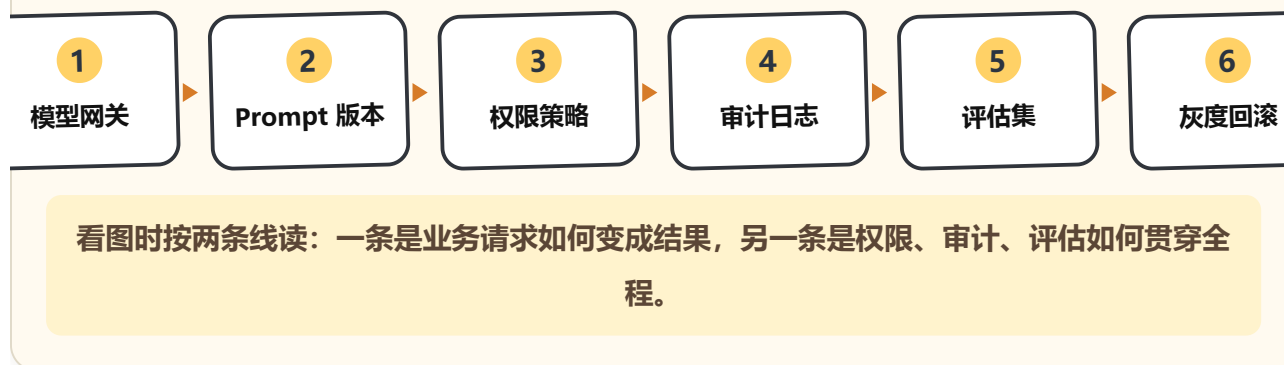
带走一个动作

把你现在项目里的一个 AI 功能画成六步链路：目标输入 -> 状态构建 -> 规划 -> Skill/Tool -> 观察 -> 终止/转人工。如果画不出来，说明这个功能现在还不是一个可治理的工程能力。

工程化与安全治理

把 AI 应用从能跑变成可上线、可追踪、可回滚、可持续优化。

工程化与安全治理 - 一张图先建立直觉



这一章先给结论

不是上线后补规范，是从第一天内置护栏。模型、Prompt、RAG、工具、Agent 都要有版本、日志、评估、预算和回滚。

工程化与安全治理：先讲一个真实卡点

AI 系统刚上线时，大家关心准确率；真正跑起来后，成本、权限、审计、灰度、回滚、注入防护都会变成生产问题。治理不是文档，是运行时机制。

模型、Prompt、RAG、工具、Agent 都要有版本、日志、评估、预算和回滚。

问题不长这样，别急着写代码

真实卡点	长什么样	工程抓手
日志缺失	只知道用户说答错了，不知道用的哪个模型和哪些证据。	记录 requestId、promptVersion、model、chunks、tools、tokens。
成本失控	长上下文、重试、流式、批量任务堆在一起。	按用户、团队、业务线、模型设置预算和限流。
只靠 Prompt 防注入	用户绕过规则或诱导工具调用。	输入隔离、知识过滤、工具权限、输出校验四层防护。

注意，这些问题没有一个是“模型太笨”能解释完的。它们通常是系统边界、数据治理、协作流程和验证手段的问题。

工程化与安全治理：知识点地图

这一页不是目录，而是本章的“问题索引”。每个知识点都要能回答：它解决什么问题，输入输出是什么，边界在哪里，失败时怎么处理。

知识点	先别背定义，先看它解决什么	这一章的落地判断
模型网关 complex	模型网关 是企业级 AI 治理能力，决定系统能否真正上线。	所有模型、Prompt、RAG 策略和工具 schema 都要版本化。
Prompt 管理 medium	Prompt 管理 是企业级 AI 治理能力，决定系统能否真正上线。	调用日志要覆盖 requestId、用户、模型、Prompt、知识、工具、token 和结果。
调用审计 complex	调用审计 是企业级 AI 治理能力，决定系统能否真正上线。	质量、成本、安全、灰度和回滚要进入发布流程。
AI 可观测性 complex	AI 可观测性 是企业级 AI 治理能力，决定系统能否真正上线。	所有模型、Prompt、RAG 策略和工具 schema 都要版本化。
质量评估 complex	质量评估 是企业级 AI 治理能力，决定系统能否真正上线。	调用日志要覆盖 requestId、用户、模型、Prompt、知识、工具、token 和结果。
成本治理 medium	成本治理 是企业级 AI 治理能力，决定系统能否真正上线。	质量、成本、安全、灰度和回滚要进入发布流程。
权限与租户 complex	权限与租户 是企业级 AI 治理能力，决定系统能否真正上线。	所有模型、Prompt、RAG 策略和工具 schema 都要版本化。
敏感信息脱敏 complex	敏感信息脱敏 是企业级 AI 治理能力，决定系统能否真正上线。	调用日志要覆盖 requestId、用户、模型、Prompt、知识、工具、token 和结果。
Prompt 注入防护 complex	Prompt 注入防护 是企业级 AI 治理能力，决定系统能否真正上线。	质量、成本、安全、灰度和回滚要进入发布流程。
灰度与回滚 complex	灰度与回滚 是企业级 AI 治理能力，决定系统能否真正上线。	所有模型、Prompt、RAG 策略和工具 schema 都要版本化。

读法：不要孤立背概念。把每一行都放回本章流程图里，看它到底卡在哪个环节。

工程化与安全治理：流程拆解图

复杂知识点要像工程流程一样拆：谁发起，谁理解，谁补事实，谁调用工具，谁执行，谁校验，谁反馈。



主流程

业务请求到结果返回，必须能复盘每一步。

补充流

知识、数据、工具、历史状态只在需要时进入上下文。

治理流

权限、审计、成本、评估贯穿全链路。

工程化与安全治理：Java 落地骨架

下面的代码不是让你照抄，而是让你抓住工程边界：模型在哪，校验在哪，审计在哪，失败怎么停。

```
Policy policy = policyEngine.check(user, task, dataScope);
AuditTrace trace = audit.start(request, promptVersion);
AiResponse res = aiGateway.callWithPolicy(policy, request);
Evaluation eval = evaluator.score(res, rules);
release.decide(eval, budget, risk);
```

落地判断标准

- 业务入口不要直接调用模型，先抽应用服务或 AI Gateway。
- 模型输出只能作为候选结果，最终进入业务系统前必须经过校验。
- 日志要能回放：用户输入、上下文、模型、知识来源、工具调用、token、耗时和结果。
- 所有高风险动作都要有人类确认或明确策略。

让生产问题的人，同时生产验证手段。否则“开发完成”只是把风险从代码里转移到了用户那里。

工程化与安全治理：学完要能自查

必须能回答

- 这个模块的输入、输出、上下游依赖分别是什么？
- 哪些事由模型负责，哪些事必须由 Java 系统负责？
- 如果资料不足、权限不足、工具失败、成本超限，系统怎么停下来？
- 怎么设计评估样本，证明这次改动真的变好？

本章小结

不是上线后补规范，是从第一天内置护栏：模型、Prompt、RAG、工具、Agent 都要有版本、日志、评估、预算和回滚。

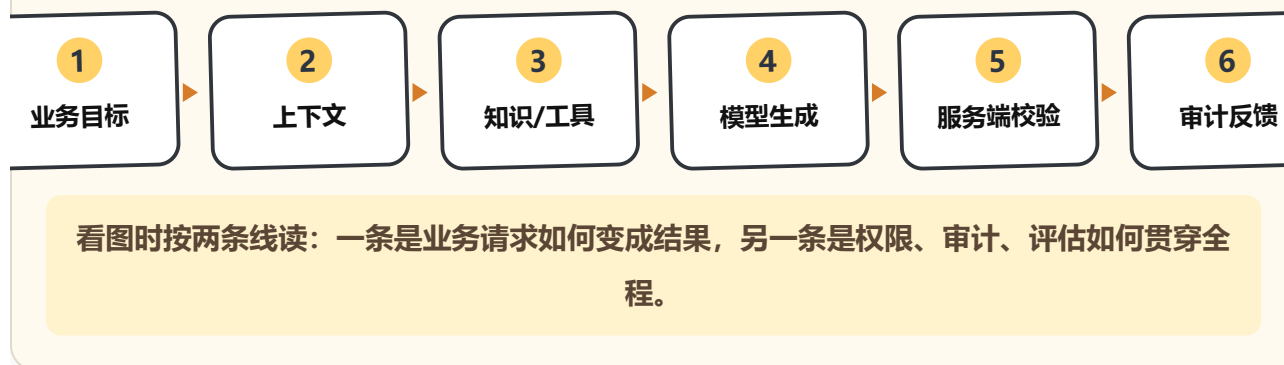
带走一个动作

把你现在项目里的一个 AI 功能画成六步链路：模型网关 -> Prompt 版本 -> 权限策略 -> 审计日志 -> 评估集 -> 灰度回滚。如果画不出来，说明这个功能现在还不是一个可治理的工程能力。

综合设计与架构表达

训练把业务目标拆成 AI 能力、数据流、控制流、风险点和验收指标的能力。

综合设计与架构表达 - 一张图先建立直觉



这一章先给结论

不是模型问题，是系统边界问题。把“会调用模型”改成“能稳定交付一个业务结果”。模型只是一段能力，工程系统才负责确定性。

综合设计与架构表达：先讲一个真实卡点

团队第一次做 AI 功能时，最容易冲进模型选型、Prompt 调参、页面 Demo。跑通以后才发现真正难的是：谁给事实、谁管权限、谁确认结果、错了谁兜底。AI 应用不是一个模型接口，而是一条业务链路。

把“会调用模型”改成“能稳定交付一个业务结果”。模型只是一段能力，工程系统才负责确定性。

问题不长这样，别急着写代码

真实卡点	长什么样	工程抓手
只画模型调用	架构图里只有前端、后端、模型，缺少知识、工具、权限、审计、评估。	补数据流和控制流，标出每个组件的输入、输出和兜底方。
目标不清	上线后很难判断到底节省了什么，质量提升在哪里。	先写业务价值假设和验收指标，再做技术方案。
失败路径缺失	资料不足也回答，工具失败也继续编，用户不满意也没有回路。	为拒答、降级、转人工、重新检索设计分支。

注意，这些问题没有一个是“模型太笨”能解释完的。它们通常是系统边界、数据治理、协作流程和验证手段的问题。

综合设计与架构表达：知识点地图

这一页不是目录，而是本章的“问题索引”。每个知识点都要能回答：它解决什么问题，输入输出是什么，边界在哪里，失败时怎么处理。

知识点	先别背定义，先看它解决什么	这一章的落地判断
从业务价值出发 medium	从业务价值出发 是 AI 项目从想法到落地的架构表达能力。	画出数据流和控制流，标出模型、知识、工具、权限、审计各自负责什么。
数据流设计 complex	数据流设计 是 AI 项目从想法到落地的架构表达能力。	先写验收指标，再决定模型/RAG/Tool/Agent 的组合。
控制流设计 complex	控制流设计 是 AI 项目从想法到落地的架构表达能力。	为拒答、降级、转人工和反馈设计明确分支。
风险点标注 complex	风险点标注 是 AI 项目从想法到落地的架构表达能力。	画出数据流和控制流，标出模型、知识、工具、权限、审计各自负责什么。
验收指标 complex	验收指标 是 AI 项目从想法到落地的架构表达能力。	先写验收指标，再决定模型/RAG/Tool/Agent 的组合。
分阶段路线 medium	分阶段路线 是 AI 项目从想法到落地的架构表达能力。	为拒答、降级、转人工和反馈设计明确分支。
团队角色 medium	团队角色 是 AI 项目从想法到落地的架构表达能力。	画出数据流和控制流，标出模型、知识、工具、权限、审计各自负责什么。
架构图表达 complex	架构图表达 是 AI 项目从想法到落地的架构表达能力。	先写验收指标，再决定模型/RAG/Tool/Agent 的组合。
汇报方式 medium	汇报方式 是 AI 项目从想法到落地的架构表达能力。	为拒答、降级、转人工和反馈设计明确分支。
后续实战准备 medium	后续实战准备 是 AI 项目从想法到落地的架构表达能力。	画出数据流和控制流，标出模型、知识、工具、权限、审计各自负责什么。

读法：不要孤立背概念。把每一行都放回本章流程图里，看它到底卡在哪个环节。

综合设计与架构表达：流程拆解图

复杂知识点要像工程流程一样拆：谁发起，谁理解，谁补事实，谁调用工具，谁执行，谁校验，谁反馈。



主流程

业务请求到结果返回，必须能复盘每一步。

补充流

知识、数据、工具、历史状态只在需要时进入上下文。

治理流

权限、审计、成本、评估贯穿全链路。

综合设计与架构表达：Java 落地骨架

下面的代码不是让你照抄，而是让你抓住工程边界：模型在哪，校验在哪，审计在哪，失败怎么停。

```
Goal goal = clarify(request);
AiBoundary boundary = splitModelKnowledgeTools(goal);
Context ctx = contextBuilder.build(request, boundary);
AiResult draft = model.generate(ctx);
CheckedResult result = verifier.check(draft, policy);
audit.record(request, ctx, result);
```

落地判断标准

- 业务入口不要直接调用模型，先抽应用服务或 AI Gateway。
- 模型输出只能作为候选结果，最终进入业务系统前必须经过校验。
- 日志要能回放：用户输入、上下文、模型、知识来源、工具调用、token、耗时和结果。
- 所有高风险动作都要有人类确认或明确策略。

让生产问题的人，同时生产验证手段。否则“开发完成”只是把风险从代码里转移到了用户那里。

综合设计与架构表达：学完要能自查

必须能回答

- 这个模块的输入、输出、上下游依赖分别是什么？
- 哪些事由模型负责，哪些事必须由 Java 系统负责？
- 如果资料不足、权限不足、工具失败、成本超限，系统怎么停下来？
- 怎么设计评估样本，证明这次改动真的变好？

本章小结

不是模型问题，是系统边界问题：把“会调用模型”改成“能稳定交付一个业务结果”。模型只是一段能力，工程系统才负责确定性。

带走一个动作

把你现在项目里的一个 AI 功能画成六步链路：业务目标 -> 上下文 -> 知识/工具 -> 模型生成 -> 服务端校验 -> 审计反馈。如果画不出来，说明这个功能现在还不是一个可治理的工程能力。

参考资料与继续深挖

- Spring AI Reference: <https://docs.spring.io/spring-ai/reference/>
- Spring AI Examples: <https://github.com/spring-projects/spring-ai-examples>
- Model Context Protocol: <https://modelcontextprotocol.io/>
- MCP Java SDK: <https://github.com/modelcontextprotocol/java-sdk>
- Microsoft Advanced RAG: <https://learn.microsoft.com/en-us/azure/developer/ai/advanced-retrieval-augmented-generation>
- OpenAI Java SDK: <https://github.com/openai/openai-java>
- LangChain4j Documentation: <https://docs.langchain4j.dev/>
- Dify: <https://github.com/langgenius/dify>
- RAGFlow: <https://github.com/infiniflow/ragflow>

资料不是为了收藏。每读一个项目或文档，都要问：它解决哪个真实卡点？它的边界是什么？它怎么验证自己是对的？